

IFT436 - Série d'exercices #3 : le tri

Manuel Lafond

Exercice 1: Considérez le célèbre tri à bulle (Bubblesort):

```
fonction triABulle( $T$ ) //  $n$  est le nombre d'éléments
  pour  $i = 1..n - 1$  faire
    pour  $j = 1..n - i$  faire
      si  $T[j] > T[j + 1]$  alors
        échanger  $T[j]$  avec  $T[j + 1]$  ;
      fin
    fin
  fin
```

Répondez aux questions suivantes:

- Démontrez que cet algorithme est correct, c'est-à-dire que le tableau est trié à la fin de l'exécution.

Indice: en vous inspirant du tri par insertion, trouvez un invariant de boucle qui est maintenu par l'algorithme. Regardez la fin du tableau par rapport à i .

Solution

Nous allons maintenir l'invariant de boucle suivant: "après l'itération i , le sous-tableau $T[n-i+1]$ est trié et $\max(T[1..n-i]) < \min(T[n-i+1..n])$ ". Si c'est vrai pour $i = n - 1$ après le dernier tour de boucle, ceci veut évidemment dire que T est trié.

Vérifions d'abord que l'invariant est vrai au premier tour avec $i = 1$. En suivant la trace de l'algorithme, on voit que lorsque $\max(T)$ est rencontré par la deuxième boucle avec un certain j , ce maximum sera décalé jusqu'à la fin de T . Ceci montre que $T[n-i+1..n] = T[n..n]$ est trié et contient le maximum.

Prenons maintenant $i > 1$. On voit que l'algorithme prend le maximum m de $T[1..n-i]$ et l'amènera à la position $n-i+1$. En supposant que l'invariant était vrai pour $i-1$, on avait que $T[n-i+1..n]$ était trié et contenait des valeurs plus grandes que m . Ceci veut dire que $T[n-i+1..n]$ est trié et, puisque m était le maximum de $T[1..n-i]$, que $T[n-i+1..n]$ contient les plus grandes valeurs, démontrant ainsi l'invariant.

- b. Quelle est le temps de cet algorithme en notation Θ , dans le pire cas?

Solution

Notons que le pire cas et le meilleur cas sont les mêmes. Peu importe le tableau, on passera toujours $n-1$ fois dans la première boucle et $n-i$ fois dans la deuxième (car rien ne permet de sortir de ces boucles avant que les itérations soient toutes effectuées). Le nombre d'instructions sera donc toujours, à une constante près, $(n-1) + (n-2) + (n-3) + \dots + (n-(n-2)) + (n-(n-1)) = \sum_{i=1}^{n-1} i \in \Theta(n^2)$.

Exercice 2: La *médiane* m d'un ensemble S de n valeurs est la valeur de S qui "sépare" en deux: la moitié des valeurs de S sont au plus m , et la moitié sont au moins m . En d'autres termes, la médiane de S est la $\lfloor n/2 \rfloor$ -ième plus grande valeur de S .

Décrivez d'abord un algorithme qui trouve la médiane de S en temps $O(n \log n)$.

Ensuite, bien que ce ne soit pas simple, il est possible de trouver la médiane

en temps $O(n)$ (n'essayez pas ça!). Montrez que ceci implique qu'on peut implémenter QuickSort en temps $O(n \log n)$ dans le pire cas.

Solution

Trouver la médiane est facile: on trie le tableau en temps $O(n \log n)$ et on retourne $T[\lceil n/2 \rceil]$.

Si on pouvait trouver la médiane en temps $O(n)$, on pourrait choisir la médiane comme pivot à chaque étape. Les sous-tableaux A et B auraient donc une taille au plus $\lceil n/2 \rceil$ et on pourrait répéter l'analyse du tri fusion. C'est-à-dire, si $t(n)$ est le temps maximum requis par l'algorithme, on peut estimer $t(n) \leq 2t(n/2) + cn$ en supposant que n est une puissance de 2 (on peut ignorer que le pivot n'est pas dans A ni dans B), avec $t(1) = 1$, et trouver que $t(n) = cn \log n + n$ par induction.

Exercice 3: Lorsqu'un nouveau maire est élu dans une ville, sa première tâche est toujours de vérifier qu'il n'y a pas de maisons qui sont trop rapprochées sur une même rue. On peut résoudre ce problème en trouvant, sur une rue donnée, la distance minimale entre deux maisons (on traite les maisons comme des points en une dimension).

Étant donné un ensemble $M = \{x_1, x_2, \dots, x_n\}$ d'entiers, on veut donc trouver $x_i, x_j \in S$ tels que $|x_i - x_j|$ est minimum parmi toutes les paires d'éléments de M . Vous pouvez représenter M comme bon vous semble, par exemple vous pouvez supposer que M est un tableau.

Donnez un algorithme qui résout ce problème. Donnez la complexité de votre algorithme avec la notation O , avec justification. Faites mieux que $O(n^2)$.

Solution

L'idée est de trier les positions en temps $O(n \log n)$. Une fois que c'est fait, la distance minimum sera entre deux points consécutifs — il ne reste qu'à les passer.

$M' = triFusion(M)$

$curmin = \infty$

$imin = \infty$

Pour $i = 2..n$

— Si $M'[i] - M'[i - 1] < curmin$

— — $curmin = M'[i] - M'[i - 1]$

— — $imin = i$

On retourne la paire x_i, x_j tels que $x_i = M'[imin]$ et $x_j = M'[imin - 1]$

Exercice 4: Soit T un tableau. On dit que les positions i et j forment une *inversion* si $i < j$ mais $T[i] > T[j]$ (donc dit autrement, si $T[i]$ et $T[j]$ sont relativement mal placés).

Reconsidérez le tri par insertion. Dans la boucle imbriquée, un *décalage* est fait lorsque l'on applique l'instruction $T[i + 1] = T[i]$. Montrez que le nombre de décalages fait par l'algorithme sur entrée T est égal au nombre d'inversions de T .

Solution

On suppose que tous les éléments de T sont distinguables (par exemple en leur donnant un identifiant même s'ils ont la même valeur). Soient $i < j$ tels que $T[i] > T[j]$. Observons d'abord que pendant le tour de boucle avant j , la valeur à $T[i]$ restera avant la position j . Puis, au tour de boucle j , il y aura nécessairement un décalage de cette valeur. Donc chaque inversion correspond à un décalage, ce qui veut dire que le nombre de décalages est au moins aussi grand que le nombre d'inversions. Mais on veut montrer l'égalité. Pour ce faire, il faut aussi argumenter que le nombre de décalages n'est pas plus grand que le nombre d'inversions. Il suffit d'observer qu'il y a un seul décalage correspond à l'inversion $T[i]$ et $T[j]$: ce décalage a lieu au tour de boucle j tel que décrit ci-haut. Ensuite, les deux valeurs ne créent plus d'inversion. Donc le nombre de décalages effectués est égal au nombre d'inversions.

Exercice 5: Rappelons qu'un algorithme est *sur-place* s'il trie un tableau T en déplaçant directement les éléments dans T . Un algorithme de tri est *stable* si, dans le tableau retourné, les éléments de valeur égale apparaissent dans le même ordre que dans le tableau donné en entrée.

Pour chacun des algorithmes suivants, dites si l'implémentation présentée dans les notes de cours est (1) sur-place, et (2) stable.

- tri par insertion
- tri fusion
- QuickSort
- triComptage
- tri radix

Solution

triInsertion : il est sur-place car ne fait que des décalages au sein du tableau donné en entrée, sans créer de tableau temporaire. Il est stable car si deux valeurs $T[i]$ et $T[j]$ sont égales, avec $i < j$, seules les valeurs strictement plus grands que $T[j]$ seront décalées vers la droite. Donc $T[i]$ restera à gauche de $T[j]$, ce qui fait que le tri est stable.

triFusion : il n'est pas sur-place, car il crée deux tableaux A et B et un tableau de retour S , qui requièrent $O(n)$ espace. Le tri fusion est toutefois stable. On peut l'argumenter par induction. Le cas de base quand $|T| = 1$ est trivial. Pour l'induction, supposons que $T[i] = T[j]$, avec $i < j$. Si $T[i]$ et $T[j]$ se retrouvent dans la même moitié de tableau, disons A , on suppose par induction que la récursion maintient l'ordre et que $T[i]$ sera avant $T[j]$ dans A . Ensuite il est facile de voir que la boucle de fusion va insérer $T[i]$ avant $T[j]$ dans S . Sinon, on suppose que $T[i]$ et $T[j]$ sont dans 2 moitiés différentes. Donc $T[i]$ est dans A et $T[j]$ est dans B . Puisque la boucle de fusion donne priorité à A , alors $T[i]$ sera inséré dans S avant $T[j]$.

quickSort : il n'est pas sur-place car il doit construire A et B et concaténer. Mais il est implémentable sur-place (voir la version sur wikipédia de quicksort). Il n'est pas stable car le pivot est toujours placé au milieu. Par exemple, avec $T = [1, 1]$, le pivot sera le premier 1. Ensuite, on aura $A = [1]$ qui contient le deuxième 1. Puis on retourne la concaténation $A + [pivot] + B$, dans quel cas le deuxième 1 se retrouve avant le premier 1. Toutefois, avec des modifications on peut implémenter quicksort de façon stable.

triComptage : il n'est pas sur place, car il crée un tableau de comptes C et un tableau de retour S . La version avec clé que j'ai présentée est stable. Pour le voir, notez que dans C , les éléments avec une clé c égale sont insérés dans $C[c]$ dans leur ordre d'apparition. Ensuite, dans S , les éléments de $C[c]$ sont insérés dans ce même ordre. Donc l'ordre relatif des éléments égaux est préservé.

Exercice 6: En réponse partielle à l'exercice précédent, la version de QuickSort présentée en classe et dans les notes n'est pas sur-place. Proposez une version sur-place du QuickSort.

Indice: pour travailler récursivement sur des sous-tableaux de T , il vous faudra deux paramètres low et hi pour les positions du sous-tableau sur lequel vous travaillez. L'en-tête de votre fonction pourrait donc avoir la forme $quickSort(T, low, hi)$.

Solution

Voir https://en.wikipedia.org/wiki/Quicksort#Lomuto_partition_scheme.

Exercice 7: Petit rappel IFT339: un arbre binaire de recherche (ABR) est une structure de données qui permet d'effectuer: l'insertion d'une valeur, la suppression d'une valeur, et la recherche de la valeur minimum. Chaque opération peut se faire en temps $O(h)$, où h est la hauteur de l'arbre (voir série exercices 2 pour définition de hauteur). On sait qu'il y a des techniques pour garantir que dans un ABR avec n éléments, on a toujours $h \in O(\log n)$ (par exemple, arbre rouge-noir, arbre AVL, B-tree, ...).

Donnez un algorithme de tri qui utilise un ABR. Quelle est la meilleure com-

plexité que vous pouvez atteindre?

Solution

Voir <https://www.youtube.com/watch?v=1kacB3mUE1E&t=2361s>.

Exercice 8: Montrez que le tri *quicksort* récursif tel que vu en classe s'exécute en temps $O(n)$.

Pour ce faire, je recommande d'argumenter comme suit:

- Observer que chaque appel à la fonction *quicksort* s'exécute en temps $O(n)$, où le n est la taille du tableau initial.
- Démontrer qu'au total, on fera au plus $2n + 1$ appels à la fonction *quicksort*, en comptant l'appel initial.

Solution

Il est facile de voir qu'un appel de *quicksort* prend un temps $O(n)$, car construire A et B ne demande qu'un parcours en temps linéaire à travers le tableau donné.

Pour montrer qu'au plus $2n + 1$ appels sont faits, on va dénoter par $f(n)$ le nombre total d'appels faits sur une entrée de taille n . On procède par induction sur n .

Comme cas de base, si $n \in \{0, 1\}$, on a $f(n) = 1$ car on retourne immédiatement. L'énoncé est vrai car $1 \leq 2n + 1$.

Pour l'induction, on suppose que l'énoncé est vrai pour des valeurs plus petites que n . Après le calcul des sous-tableaux A et B , soit $n_1 = |A|$ et $n_2 = |B|$. On a

$$f(n) = 1 + f(n_1) + f(n_2)$$

car on compte l'appel courant, en plus de ceux faits par les deux appels récursifs. Par hypothèse d'induction, on a

$$f(n) \leq 1 + 2n_1 + 1 + 2n_2 + 1 = 3 + 2(n_1 + n_2).$$

Ensuite, on peut déduire que $n_1 + n_2 = n - 1$, parce que l'algorithme met toutes les valeurs du tableau d'entrée dans A ou dans B , sauf le pivot. On déduit que

$$f(n) \leq 1 + 2(n - 1) + 2 = 1 + 2n - 2 + 2 = 2n + 1.$$

L'algorithme fait donc au plus $2n + 1$ appels, chacun en temps $O(n)$, et le temps total est donc $O(n(2n + 1)) = O(n^2)$.